

Lecture 06: RSA & Diffie-Hellman Key Exchange

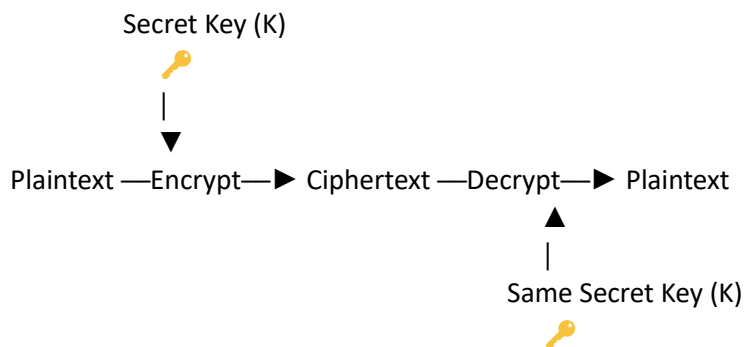
Introduction to Communication Security

Communication security means protecting information while it is being sent from a sender to a receiver. The original readable message is called plaintext. After encryption, it becomes ciphertext, which cannot be understood without the correct method. After decryption, it becomes the original plaintext again.

A) Secret Key (Symmetric) Cryptography

SKC uses a single key for both encryption and decryption.

Diagram



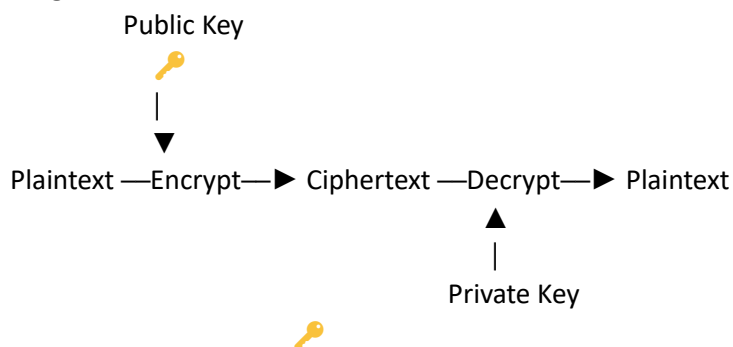
Explanation

In symmetric cryptography, only one key is used. The sender uses this key to encrypt the plaintext into ciphertext. The receiver uses the same key to decrypt the ciphertext and recover the original plaintext. Therefore, both the sender and the receiver must know the same secret key and keep it private.

B) Public Key (Asymmetric) Cryptography

PKC uses two keys, one for encryption and the other for decryption.

Diagram



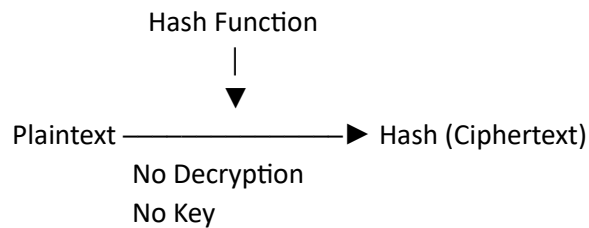
Explanation

In asymmetric cryptography, two different keys are used. The public key is used to encrypt the plaintext, while the private key is used to decrypt the ciphertext. Since the encryption key and decryption key are different, only the owner of the private key can recover the original plaintext.

C) Hash Function (One-Way Cryptography)

Hash functions have no key because the plaintext cannot be recovered from the ciphertext (hash).

Diagram



Explanation

A hash function converts plaintext into a hash value (hash). It does not use any key. The process is one-way, which means the original plaintext cannot be recovered from the hash. Therefore, hash functions are called one-way cryptography.

RSA Algorithm

RSA (Rivest-Shamir-Adleman) is the most widely used public-key cryptographic algorithm. It was invented in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman.

RSA uses two different keys:

- **Public Key** → Used for encryption
- **Private Key** → Used for decryption

Unlike symmetric encryption, the encryption key and decryption key are different.

RSA provides:

- Confidentiality (Encryption)
- Authentication (Digital Signature)
- Integrity
- Non-repudiation

Basic Idea of RSA

Suppose Alice wants to receive secure messages.

She creates:

- Public Key → Shared with everyone
- Private Key → Kept secret

Bob encrypts the message using Alice's public key. Only Alice can decrypt it using her private key. Therefore, even if an attacker intercepts the ciphertext, they cannot recover the original message without the private key.

RSA Keys

Public Key

Contains:

$$(e, n)$$

where

- **e** = public exponent
- **n** = modulus

Everyone knows this key.

Private Key

Contains:

$$(d, n)$$

where

- **d** = private exponent

Only the owner knows this key.

RSA Key Generation

Generating RSA keys consists of several mathematical steps.

Step 1: Choose Two Prime Numbers

Choose two large prime numbers. Example:

$$p = 3, q = 11$$

Step 2: Compute n

Multiply both primes.

Formula:

$$n = p \times q$$

Example

$$n = 3 \times 11 = 33$$

Step 3: Compute Euler's Totient

Formula

$$\phi(n) = (p - 1)(q - 1)$$

Example

$$\begin{aligned}\phi(33) &= (3 - 1)(11 - 1) \\ &= 2 \times 10 = 20\end{aligned}$$

Step 4: Choose Public Exponent e

Choose a number satisfying

$$1 < e < \phi(n)$$

and

$$\gcd(e, \phi(n)) = 1$$

Example

Choose

$$e = 7$$

Since

$$\gcd(7, 20) = 1$$

it is acceptable.

Step 5: Compute Private Exponent d

Choose d such that

$$e \times d \equiv 1 \pmod{\phi(n)}$$

Example

$$7 \times 3 = 21$$

Since

$$21 \equiv 1 \pmod{20}$$

therefore

$$d = 3$$

Final Keys

Public Key

$$(7, 33)$$

Private Key

$$(3, 33)$$

RSA Encryption

Encryption converts plaintext into ciphertext.

Formula

$$C = M^e \pmod{n}$$

where

- M = plaintext
- C = ciphertext
- e = public exponent
- n = modulus

Encryption Example

Plaintext

$$M = 2$$

Public key

$$(7, 33)$$

Then

$$\begin{aligned} C &= 2^7 \pmod{33} \\ 128 \pmod{33} &= 29 \end{aligned}$$

Ciphertext

$$C = 29$$

RSA Decryption

Decryption converts ciphertext back into plaintext.

Formula

$$M = C^d \pmod{n}$$

where

- C = ciphertext
- d = private exponent

Example from Lecture

Ciphertext

$$C = 4$$

Private Key

$$(3, 33)$$

Then

$$\begin{aligned} M &= 4^3 \pmod{33} \\ 64 \pmod{33} &= 31 \end{aligned}$$

Recovered message

$$M = 31$$

Why RSA is Secure?

RSA security depends on the difficulty of **integer factorization**.

The public key contains only

$$(e, n)$$

where

$$n = p \times q$$

An attacker knows n but does not know p and q . Finding p and q from a very large n (typically **2048 bits**) is computationally infeasible using current technology.

Without p and q :

- Euler's Totient cannot be computed.
- Private key d cannot be calculated.
- Ciphertext cannot be decrypted.

Therefore RSA remains secure.

Breaking RSA

If an attacker somehow factors

$$n = p \times q$$

they can calculate

Step 1

$$\phi(n) = (p - 1)(q - 1)$$

Step 2

Find d satisfying

$$e \times d \equiv 1 \pmod{\phi(n)}$$

Now the attacker has the private key. Therefore factoring n completely breaks RSA.

Advantages of RSA

- Very secure
- Public key can be shared openly
- Supports digital signatures
- Supports authentication
- Widely used on the Internet
- Used in SSL/TLS, HTTPS, Email Security, VPNs

Disadvantages of RSA

- Slower than symmetric encryption
- Requires large key sizes
- Computationally expensive
- Not suitable for encrypting very large files directly
- Vulnerable if weak keys are used

Applications of RSA

- HTTPS
- SSL/TLS
- Secure Email
- VPN
- Digital Certificates
- Digital Signatures
- Secure Banking

Important RSA Formulas

Key Generation

$$\begin{aligned}n &= p \times q \\ \phi(n) &= (p - 1)(q - 1) \\ ed &\equiv 1 \pmod{\phi(n)}\end{aligned}$$

Encryption

$$C = M^e \bmod n$$

Decryption

$$M = C^d \bmod n$$

Diffie-Hellman Key Exchange

Diffie-Hellman Key Exchange (DH) is the first practical **public-key key exchange** algorithm, introduced by Whitfield Diffie and Martin Hellman in 1976. Its purpose is not encryption, but to allow two parties to securely establish a shared secret key over an insecure communication channel. This shared secret can then be used with symmetric encryption algorithms such as AES.

Main Idea

Suppose Alice and Bob want to communicate securely.

Initially:

- They have no shared secret.
- An attacker can listen to every message.

Using Diffie-Hellman, they independently compute the same secret key without ever transmitting the key itself. Even though an attacker sees all exchanged values, deriving the secret key is computationally infeasible.

Public Parameters

Both users agree publicly on:

- Large prime number
- Primitive root

$$q$$

$$\alpha$$

These values are public.

Private Keys

Alice chooses

$$a$$

Bob chooses

$$b$$

These values remain secret.

Public Keys

Alice computes

$$Y_A = \alpha^a \bmod q$$

Bob computes

$$Y_B = \alpha^b \bmod q$$

Then they exchange these values.

Shared Secret

Alice computes

$$K = Y_B^a \bmod q$$

Bob computes

$$K = Y_A^b \bmod q$$

Both obtain exactly the same key.

Diffie-Hellman Example

Given

$$q = 23$$

$$\alpha = 5$$

Alice chooses

$$a = 6$$

Bob chooses

$$b = 15$$

Alice Sends

$$Y_A = 5^6 \bmod 23$$
$$15625 \bmod 23 = 8$$

Alice sends **8**.

Bob Sends

$$Y_B = 5^{15} \bmod 23$$

Result

$$19$$

Bob sends **19**.

Shared Secret

Alice computes

$$19^6 \bmod 23 = 2$$

Bob computes

$$8^{15} \bmod 23 = 2$$

Final secret key

$$K = 2$$

Diffie-Hellman Security

The shared secret key is never transmitted over the network. Only intermediate public values are exchanged.

An attacker knows

- q
- α
- Public values

but cannot efficiently determine the private values because of the Discrete Logarithm Problem.

Discrete Logarithm Problem

Formula

$$Y = \alpha^X \bmod q$$

Given

- Y
- α
- q

Finding X is computationally infeasible when q is very large. This computational hardness is called the Discrete Logarithm Problem (DLP). It forms the security foundation of Diffie-Hellman.

Advantages of Diffie-Hellman

- Secure key exchange over insecure networks
- Shared secret is never transmitted
- Simple mathematical design
- Widely used in TLS, SSH, VPNs, IPsec
- Foundation of many modern protocols

Limitations of Diffie-Hellman

1. Computationally Expensive

Requires modular exponentiation, which is slower than simple arithmetic operations.

2. Vulnerable to DoS Attacks

Attackers can send many fake key exchange requests, forcing expensive computations and consuming server resources.

3. Cannot Encrypt Data

Diffie-Hellman only establishes a shared secret key. It does **not** encrypt messages. A symmetric algorithm (e.g., AES) must be used afterward.

4. No Authentication

Diffie-Hellman does not verify the identity of communicating parties. An attacker can impersonate users, making the protocol vulnerable to Man-in-the-Middle attacks.

Bucket Brigade Attack (Man-in-the-Middle Attack)

A Man-in-the-Middle (MITM) Attack, also called the Bucket Brigade Attack, occurs when an attacker secretly intercepts and relays communication between two parties while making each believe they are communicating directly.

How It Works

1. Alice sends her public value to Bob.
2. Eve intercepts the message.
3. Eve sends her own public value to Bob instead.
4. Bob replies with his public value.
5. Eve intercepts it and sends her own value to Alice.
6. Alice establishes a secret with Eve.
7. Bob also establishes a different secret with Eve.
8. Eve decrypts, reads, modifies (if desired), re-encrypts, and forwards every message.

Thus, Alice and Bob never share the same secret; each unknowingly shares a key with Eve.

Why MITM is Possible in Diffie-Hellman

- No authentication mechanism.
- Public values are not digitally signed.
- The protocol cannot verify the sender's identity.

Prevention

To prevent MITM attacks:

- Digital Signatures
- Digital Certificates (PKI)
- Authentication Protocols
- Authenticated Diffie-Hellman
- TLS/SSL Certificates
- Certificate Authorities (CA)

RSA vs Diffie-Hellman

Feature	RSA	Diffie-Hellman
Purpose	Encryption, Digital Signature	Key Exchange
Public Key	Yes	Yes
Private Key	Yes	Yes
Encryption	Yes	No
Digital Signature	Yes	No
Shared Secret	No	Yes
Authentication	Yes (with signatures)	No (basic DH)
Security Based On	Integer Factorization Problem	Discrete Logarithm Problem